Week 2 – Day 1 – Assignment

Test Document Plan based on chain-of-thought method:-

Instruction:-
Context:-
-As a test lead give me a test strategy document and plan for a mobile application
Instruction:-
-Document should be included Scope, Testing plan, App Behavior and Reporting.
-[IMPORTANT] Ask me 10 questions on this and follow step by step procedure once after proceeding with next question

Before I create a tailored **Test Strategy Document + Plan**, I'll follow your instruction and proceed step-by-step by asking questions.

✅ Step 1 – Question 1 of 10

Q1. What type of mobile application are we testing?
(e.g., Banking, E-commerce, Healthcare, Social Media, Gaming, Enterprise app)

👉 This helps define risk areas, compliance needs, and testing depth.

Please answer this, and I'll move to **Question 2**.

Banking

Great — a **banking app** means we'll need strong focus on security, compliance, and reliability.

✅ Step 2 – Question 2 of 10

Q2. Which platforms does the application support?

- Android
- iOS
- Both (Native / Hybrid / Cross-platform like Flutter, React Native)

👉 This impacts device coverage, automation tools, and compatibility strategy.

Please answer, and I'll proceed to **Question 3**.

Both

Got it — **both Android and iOS** 👍

✅ Step 3 – Question 3 of 10

Q3. What are the key features of the banking app?
(Select or list the main ones)

- User login (PIN / Password / Biometrics)
- Fund transfer (NEFT/IMPS/UPI)
- Account balance & statements
- Bill payments / recharge
- Card management
- Notifications / alerts
- Others?

👉 This helps prioritize **critical test scenarios** and risk areas.

Please share the features, and I'll move to **Question**

User login (PIN / Password / Biometrics)
Fund transfer (NEFT/IMPS/UPI)
Account balance & statements

Perfect — these are **core banking features**, so we'll treat them as **high-risk, high-priority areas**.

✅ Step 4 – Question 4 of 10

Q4. What authentication methods are used?
(You can choose multiple)

- OTP (SMS / Email)
- PIN
- Password
- Biometrics (Fingerprint / Face ID)
- Multi-Factor Authentication (MFA)
- Device binding (trusted device)

👉 This is critical for defining **security testing, negative scenarios, and compliance checks**.

Please confirm, and I'll proceed to **Question 5**.

PIN

Got it — **PIN-based authentication only**. That simplifies flow but increases the need for strong **security and brute-force protection testing**.

✅ Step 5 – Question 5 of 10

Q5. Does the app require internet connectivity for all operations, or are there any offline capabilities?

- Fully online (requires internet always)
- Partial offline (e.g., view cached data)
- Offline mode supported

👉 This impacts **network testing, caching validation, and failure scenarios**.

Please answer, and I'll proceed to **Question 6**.

Fully Online

Great — **fully online app**. That means we'll emphasize **network reliability, API stability, and failure handling**.

✅ Step 6 – Question 6 of 10

Q6. What environments are available for testing?

- Dev
- QA / Test
- UAT
- Production (limited validation)

👉 This helps define **test cycles, data strategy, and release confidence**.

Please share the environments, and I'll proceed to **Question 7**.

Dev
QA / Test
UAT

Perfect — having **Dev, QA/Test, and UAT** environments gives us a solid release pipeline 👍

✅ Step 7 – Question 7 of 10

Q7. What level of test automation is expected?

- No automation (manual only)
- Partial automation (smoke / regression)
- High automation (CI/CD integrated, end-to-end automation)

👉 This will shape the **testing plan, effort estimation, and tool strategy**.

Please answer, and I'll proceed to **Question 8**.

High automation (CI/CD integrated, end-to-end automation)

Excellent — **high automation with CI/CD** is ideal for a banking app 👍

✅ Step 8 – Question 8 of 10

Q8. Are there any regulatory or compliance requirements?
(Important for banking apps)

- RBI guidelines (India)
- PCI-DSS (for card transactions)
- GDPR / Data privacy laws
- Internal security standards
- Not sure / To be defined

👉 This directly impacts **security testing, audit readiness, and documentation rigor**.

Please confirm, and I'll proceed to **Question 9**.

RBI guidelines (India)

Great — **RBI compliance** is critical for banking apps in India. We'll include **strict security, audit trails, and data protection validation** in the strategy.

✅ Step 9 – Question 9 of 10

Q9. What is the expected user load or scale of the application?

- Low (≤ 10K users)
- Medium (10K – 1M users)
- High (1M+ users)
- Not defined

👉 This helps define **performance testing, load testing, and scalability strategy**.

Please answer, and I'll proceed to the **final Question 10**.

Low (≤ 10K users)

Great — **low user scale (≤10K users)**, but since it's a banking app, we'll still treat it as **high-risk in security and reliability**, not low priority.

✅ Step 10 – Question 10 of 10

Q10. What is the release strategy for the application?

- Big bang release (all users at once)
- Phased rollout (by region / user group)
- Pilot release (small internal/external users first)
- Continuous deployment (frequent small releases)

👉 This affects **test coverage depth, regression cycles, and UAT strategy**.

Please answer, and once done I will generate the complete **Test Strategy Document + Test Plan** with all required sections:

✔ Scope
✔ Testing Plan
✔ App Behavior

TEST STRATEGY DOCUMENT & TEST PLAN

Mobile Banking Application (Android & iOS)

---

1. 📌 SCOPE

1.1 In Scope

The testing scope includes validation of the following modules:

🔐 **Authentication**

- **PIN-based login**
- **Session management**
- **Auto-lock and timeout behavior**

💰 **Core Banking Features**

- **Account balance inquiry**
- **Account statement generation**
- **Fund transfers (NEFT / IMPS / UPI simulation as applicable)**

📡 **System Integration**

- **API validation with backend services**
- **Real-time transaction updates**
- **Error handling for service failures**

📱 **Platform Coverage**

- **Android (multiple OS versions)**
- **iOS (multiple OS versions)**
- **Different screen sizes and device classes**

🔁 **Release Model**

- **Continuous deployment (frequent releases)**
- **Regression testing per build**

1.2 Out of Scope

- **Third-party banking network failures beyond simulation**
- **Rooted/jailbroken device support (unless required)**
- **External financial institution systems beyond API contract**

1.3 Compliance Scope

- **RBI guidelines (India):**
  - **Secure authentication practices**
  - **Data encryption in transit and at rest**
  - **Audit logging for all financial transactions**
  - **Session security and timeout enforcement**

---

2. □ TESTING PLAN

2.1 Testing Approach

We follow a risk-based + continuous testing strategy aligned with CI/CD.

🔄 **Testing Layers**

- **Unit Testing (Dev-owned)**
- **API Testing (automation-first)**
- **Integration Testing (backend + mobile app)**
- **System Testing (end-to-end flows)**
- **Regression Testing (per CI/CD build)**
- **UAT (business validation in controlled environment)**

2.2 Test Types

◈ Functional Testing

- PIN login validation
- Fund transfer flows
- Balance & statement accuracy
- Transaction status updates

◈ API Testing (Critical)

- Authentication APIs
- Transaction APIs
- Statement generation APIs
- Error response validation

◈ Security Testing (HIGH PRIORITY)

- PIN brute-force attempt handling
- Session hijacking prevention
- API token validation
- Secure storage (no sensitive data in logs/cache)
- Encryption validation (TLS 1.2+)

◈ Compatibility Testing

- Android versions fragmentation
- iOS version support matrix
- Device performance variations

◈ Network Testing (Fully Online App)

- No internet scenario handling
- Intermittent connectivity
- Slow network simulation
- API timeout handling

◈ Regression Testing (CI/CD Driven)

- Automated regression suite triggered per build
- Smoke suite for quick validation
- Full regression before UAT release

2.3 Automation Strategy (High Automation)

□ Tools

- Appium (Cross-platform UI automation)
- REST Assured / Postman (API automation)
- Jenkins / GitHub Actions (CI/CD pipeline)
- TestNG / JUnit frameworks

🤘 Automation Coverage

- Smoke Tests → 100% automated
- Regression Tests → 80–90% automated
- Critical user journeys → fully automated
- Negative scenarios → partially automated

2.4 Entry / Exit Criteria

Entry Criteria

- Build deployed to QA
- APIs stable and accessible
- Test data available
- Automation suite updated

Exit Criteria

- 100% critical test cases passed
- No open Critical/High defects
- Regression suite passed in CI/CD
- Security checks cleared
- QA sign-off completed

---

3. 📱 APP BEHAVIOR (EXPECTED)

3.1 Login Behavior (PIN-based)

- App must lock after multiple failed attempts
- Session timeout on inactivity
- Secure PIN entry (masked input)
- No PIN stored locally in plain format

3.2 Transaction Behavior

Fund Transfer (NEFT / IMPS / UPI)

- Transaction must be atomic (no partial success)

- **Real-time status updates required**
- **Duplicate transaction prevention**
- **Proper error messaging for failures**

3.3 Account Services

- **Balance should reflect real-time backend data**
- **Statement generation must be consistent and accurate**
- **No caching of sensitive financial data beyond session**

3.4 Network Behavior (Fully Online App)

- **App should gracefully handle:**
  - **No internet → user-friendly error screen**
  - **Slow internet → loading indicators**
  - **API timeout → retry mechanism**
- **No transaction should be committed without confirmation response**

3.5 Security Behavior (RBI Compliance)

- **End-to-end encryption for all requests**
- **Session expiry enforced strictly**
- **Logs must not contain sensitive data**
- **Device binding recommended (if applicable in future scope)**

---

4. 📊 REPORTING STRATEGY

4.1 Daily Reporting (QA Dashboard)

- Number of test cases executed
- Pass / Fail rate
- Open defects by severity
- Automation execution results (CI/CD pipeline)

4.2 Defect Reporting

Severity Levels

Severity	Definition
Critical	App crash, transaction failure, security breach
High	Major functional failure (login/transfer issues)
Medium	UI/functional inconsistency
Low	Cosmetic issues

4.3 Test Summary Report (Per Release)

Includes:

- Scope covered
- Test execution metrics
- Defect summary
- Automation coverage report
- Risk assessment

- **Go / No-Go recommendation**

4.4 CI/CD Reporting

- **Automated test results per build**
- **Regression suite status**
- **API health check results**
- **Deployment validation status**

4.5 UAT Reporting

- **Business validation feedback**
- **Sign-off status**
- **Known issues list**
- **Release readiness confirmation**

---

5. ⚠ KEY RISKS & MITIGATION

Risk	Mitigation
API downtime	Mock services + retries
Transaction inconsistency	Strong integration + reconciliation tests
Security vulnerabilities	Regular penetration testing
Device fragmentation	Device cloud testing
CI/CD instability	Stable regression suite + gating

---

6. 🚀 FINAL STRATEGY SUMMARY

- **Type: Banking Mobile App (High Risk)**
- **Platforms: Android + iOS**
- **Auth: PIN-based**
- **Architecture: Fully online**
- **Testing Model: Risk-based + CI/CD driven**
- **Automation: High (80–90%)**
- **Compliance: RBI guidelines**
- **Release Model: Continuous deployment**